

Software agent technologies

H S Nwana and M Wooldridge

It is by now a cliché that there is no one, universally accepted definition of intelligent agent technology, but a number of loosely related techniques. Yet there are certain themes that appear common to agent-based systems, and, correspondingly, certain problems that must be addressed and overcome by all agent system builders. The aim of this paper is to briefly survey the tools and techniques that can be used to address these common issues, and that hence form a substrate for software agent systems. This paper begins with a review of agent communication languages, focusing particularly on the emerging standard known as KQML. Then a thumbnail sketch of various programming languages for building agent-based systems is presented, and there follows a discussion on support for ontologies, which allow agents to communicate using commonly defined terms and concepts. Then other computing infrastructure support for agent-based systems is considered, in particular, the use of client/server architectures and distributed object frameworks. Finally, some general comments and conclusions are presented.

1. Introduction

At the time of writing, intelligent agents and multi-agent systems are among the most rapidly growing areas of research and development in computer science. Unfortunately, as with object technology a decade ago, there is great confusion about agents — hardly anyone agrees on even the basic question of what an agent is, still less on more contentious issues. New definitions and systems seem to appear one week, and disappear the next. Journalists and practitioners, ever eager to keep abreast of new technologies, attend conferences and seminars only to be presented with a collection of apparently unrelated concepts and terms; and, of course, experience with all things object-oriented indicates that this situation is here to stay, at least for the foreseeable future.

Given this state of affairs, it would be naive to make predictions about how agent technology will develop, and, in particular, about the exact form or structure of future agent applications. Yet there are certain themes that appear common to most agent-based systems, and, correspondingly, certain problems that must be addressed and overcome by all agent system builders. In brief, the purpose of this paper is to survey the various technologies that are capable of providing (or which currently provide) this substrate for building agent-based applications. It should be noted that, for the reasons mentioned above, no attempt is made to survey actual agent architectures or applications. Rather, the aim is to focus on what might be called the ‘enabling technologies’ for agent-based systems.

In order to facilitate such an overview, an abstraction of some current or future technologies involved is helpful. Figure 1 shows one classification of some of the technologies involved. It is emphasised that this represents an abstract organisation of the technologies that facilitate the implementation of agent-based applications [1]. The hierarchy depicted may be considered by some to be contentious and arbitrary. Nevertheless, it represents a useful classification device, around which the remainder of the paper is structured.

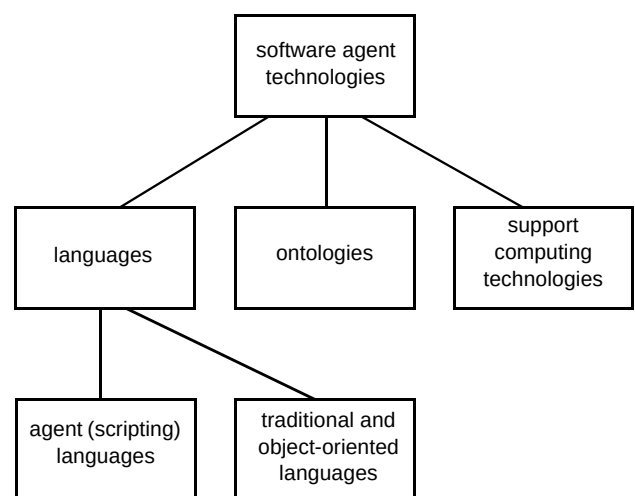


Fig 1 A classification of some technologies for developing software agent applications.

Typically, applications containing multiple agents make use of an agent communication language (ACL); the idea is similar to a human society using a common language (such as English). A couple of categories of agent communication languages are reviewed in section 2. In section 3, languages that facilitate the construction of agent applications are briefly listed. However, it is noted that agent-based applications can be (and have been) developed using traditional third-generation languages like Lisp, C, or Prolog, and object-oriented (OO) languages such as C++ or Smalltalk. Next, agents working together need to share a certain amount of foundational, ‘common’ knowledge, in just the same way that humans do. When one agent tells another agent to buy a widget, they must agree both on what a ‘widget’ is, and what ‘buy’ means. Section 4 reviews the techniques available for representing such foundational knowledge. In section 5, some current and emerging computing technologies that lend themselves to supporting agent-based applications are briefly discussed, and section 6 provides a discussion and conclusion to the paper.

2. Agent communication languages

A key rationale for having multiple agent systems (MAS) is that the ensemble of agents provide ‘added value’, beyond that which would otherwise be obtained from any single agent [2, 3]. This added value is typically achieved via co-operation. If every agent had perfect information about the state of the entire system — if every agent knew what every other agent knew, what every other agent intended to do, and how everything in the system stood in relation to everything else in the system — then co-operation could proceed without any communication at all [4]. The agents could work together as a perfectly co-ordinated team, without ever having to communicate. Of course, except in the most trivial systems, such perfect knowledge is impossible to achieve. Agents have at best partial, possibly incorrect information about the state of their environment. Thus, in order to co-operate effectively in any moderately realistic system, agents are required to communicate with one another. Where agents need to communicate, they must individually ‘understand’ some agent communication language (ACL). Indeed, Genesereth and Ketchpel [5] maintain that some piece of software:

“...is a software agent if, and only if, it communicates correctly in an agent communication language.”

Although this statement is arguable because it fails to mention other attributes that are considered germane to agenthood, the importance of having an ACL is acknowledged — imagine a society without some *lingua franca*.

ACLs are designed specifically to facilitate communication between two or more agents. Agents need

to communicate information, intentions, goals, etc, to other agents. As Cohen and Levesque [6] also note, an ACL should allow agents to enlist the support of others to achieve goals, to monitor their execution, to report progress, success, failure, to acknowledge receipt of messages, to refuse task allocations, and to commit to performing tasks for other agents.

Most ACLs, both ‘standard’ and *ad hoc*, derive their inspiration from speech act theory [7, 8] which was developed by linguists in an attempt to understand how humans use language in everyday situations, to achieve everyday ends. In speech act theory, human utterances are viewed as actions, in the same sense as actions performed in the everyday physical world (e.g. picking up a block from a table). The theory considers three aspects of utterances. Locution refers to the act of utterance itself — simply uttering an acceptable sentence of some language. Illocution refers to the ‘type’ of utterance, for example, an utterance may be a request to turn on the heater or an assertion about the temperature. There are many different illocutionary verbs (or performatives) in English — examples include ‘request’, ‘warn’, ‘inform’. Perlocution refers to the effect of an utterance — how it influences the recipient. In the context of agent-based applications, researchers have proposed ACLs based on speech act theory, or more specifically, on illocutionary speech acts.

The illocutionary verbs (such as request, inform, warn) in a natural language like English typically correspond to performatives in some ACL. There is an implicit assumption that the illocutionary performative utterance of some agent succeeds because the sending agent is communicating some attitude(s) of itself such as beliefs, goals, or assertions. It is illocutionary because the agent expects that its utterance will be understood (as it intends it to be) by the receiving agent, even though it has no control over how the utterance influences this receiving agent, i.e. it is not perlocutionary. The aspect of locution, in the context of ACLs, is trivial because it is usually implicit in the explicitly encoded performative. In other words the type and structure of the message, for example:

```
accept [to: agent_2, from: agent_3, reference: contract_4,
cost: £40]
```

is implicitly an instance of a locutionary act. More importantly, it is assumed that by virtue of being sent, it will effect some illocutionary action (in this example, agent_2 updates its belief set to reflect the fact that agent_3 has accepted to do contract_4 at the cost of £40).

For the purposes of this paper, ACLs will be classified into two categories — ‘standard’ ones and *ad hoc* ones.

2.1 A standard ACL — KQML

The Knowledge Query and Manipulation Language (KQML) is an evolving standard ACL, being developed as part of the DARPA knowledge-sharing effort (KSE) [9, 10]. The KSE is a distributed research program that is investigating and constructing software tools for co-ordination and knowledge sharing among information systems, i.e. between separate knowledge-based modules, as well as between knowledge-based systems and databases [11]. As Finin [12] notes, the KSE research program:

“...focuses on the ability of such agents to effectively interoperate by communicating information and knowledge in spite of problems introduced by heterogeneity of platforms, implementation technology, operating environments and development. This communication requires a common language or language framework which involves syntactic, semantic and pragmatic components.”

The KSE programme consists of four separate but inter-working groups. One major product so far from one of these groups, the External Interfaces Working Group (EIWG), is KQML — a high-level communication language and protocol for exchanging information and knowledge. KQML has been implemented in several research groups across North America.

At the heart of KQML are more than three dozen performatives that define the allowed ‘speech acts’ that agents may use, and which provide the substrate for constructing more complex co-ordination and negotiation strategies (as discussed in Nwana et al [4]). These performatives are grouped into nine categories, as shown in Table 1.

The KQML language can be viewed as consisting of three layers — the content, message and communication layers, as shown in Fig 2.

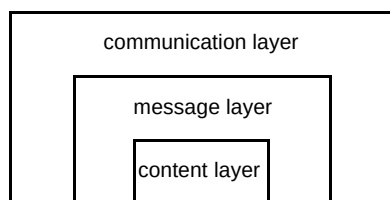


Fig 2 An abstract view of the KQML language.

The content layer specifies the actual content of the message, and the KQML standard itself has nothing to say about this. A KQML-conforming message could contain Prolog code, C code, natural-language expressions, or whatever, as long as it is ASCII-representable. The set of performatives provided by the language, and shown in

Table 1, constitute the message layer, which in turn forms the core of the language. This layer of abstraction provides the performative and specifies the protocol for delivering the message that subsumes the content.

Table 1 KQML performatives [13].

Category	Reserved performative names
Basic informational performatives	tell, deny, untell, cancel,
Basic query performatives	evaluate, reply, ask-if, ask-about, ask-one, ask-all, sorry
Multi-response query performatives	stream-about, stream-all
Basic effector performatives	achieve, unachieve
Generator performatives	standby, ready, next, rest, discard, generator
Capability definition performatives	advertise
Notification performatives	subscribe, monitor
Networking performatives	register, unregister, forward, broadcast, pipe, break
Facilitation performatives	broker-one, broker-all, recommend-one, recommend-all, recruit-one, recruit-all

Protocol is defined as the rules that agents must use when initiating and maintaining an exchange. KQML specifies several protocols, including synchronous (where a blocking query waits for an expected reply) and asynchronous (which involves non-blocking messages).

The chosen performative specifies that the content is a query, an assertion or any of the other categories of performatives. The set of KQML performatives is extensible, but EIWG has tried to strike a balance between providing a small set (thereby requiring overloading at the content layer), and providing a large set (where they are likely to overlap with one another, or else the distinctions between them become very fine). These performatives are neither necessary nor sufficient for all agent applications; however, agent application developers are encouraged to use them as specified in order to increase interoperability across applications. If some application does not support the correct usage of any of the performatives, then it would not be KQML-compliant.

The communication layer encodes low-level communication parameters, such as the identities of the sender and the recipient, and unique identifiers for the particular speech act. It should be noted that the provision of a secure and reliable communications medium (essentially, the lower five layers of the seven-layer OSI model) tends to be taken for granted in most agent-related research. Only the upper two layers (presentation and application) are considered an issue.

For a flavour of KQML, here is an example of an instantiated performative:

```
(tell
: content "cost(bt, service-4, £5677)"
: language standard prolog
: ontology bt-services-domain
: in-reply-to quote service-4
: receiver customer-2
: sender bt-customer-services)
```

The KQML performative in this message is *tell*, and the agent that is sending it seeks to inform some customer, *customer-2*, of a quote for performing some service, *service-4*, in reply to an earlier request from *customer-2*, to BT customer services. The content of the message is expressed in standard Prolog, and the ontology for BT's services domain is assumed.

KQML has a number of shortcomings. Cohen and Levesque [6] identify three general difficulties with it. Firstly, the actual specification of KQML is ambiguous and vague. KQML does not, in fact, have a precise, formal semantics, as is normal of programming languages. Cohen and Levesque believe this is a real problem:

"Without [a precise semantics], agent designers cannot be certain that the interpretation they are giving to a 'performative' is in fact the same as the one some other designer intended it to have. Moreover, the lack of a semantics for communication acts leads to a number of confusions in the set of reserved 'performatives' supplied." [6]

In KQML's defence, its inventors emphasise that it:

"...is concerned primarily with pragmatics (and secondarily with semantics)." [10]

Pragmatics, as Huhns [14] explains, is about knowing with whom to communicate and how to find them, knowing how to initiate and maintain an exchange (i.e. protocol), and knowing the effect of the communication on the recipient.

More troubling, it is not at all clear how one could go about giving the language a precise semantics, in such a way that it is possible to determine whether any given application is or is not KQML-conformant. Also, if it is impossible to determine, accurately and unambiguously, whether or not a system that claims to conform to some standard actually does conform to the standard, one must ask what the point of the so-called standard is. Secondly and thirdly, Cohen and Levesque also argue [6] that KQML suffers from mis-identified and missing performatives respectively, i.e. they suggest that some KQML performatives are not what they claim to be, and that, in addition, KQML is missing an entire class of performatives

(commissives, which commit the utterer to a course of action).

Mayfield et al [1] also attempt an evaluation of KQML, and highlight some of its other merits and demerits. On the positive side, many prototype applications have already been constructed in North America and Europe that are KQML-compliant. It may be that, in the absence of any serious competition, KQML is becoming the *de facto* ACL standard.

2.2 *Ad hoc* ACLs

To date, many agent-based applications with collaborative agents use an *ad hoc* set of performatives within *ad hoc* agent communication languages. Many others, strictly speaking, do not have explicit ACLs — they communicate by depositing information in some shared data structure. Applications with their own ACLs are mostly speech act-based. They possess a limited subset of performatives, similar to KQML's, but they are usually specified differently to their KQML equivalents, and they have different protocols. For example, SRI's Open Agent Architecture [15] has an ACL called the Inter-agent Communication Language (ICL). It possesses only three speech act types — solve (i.e. a question), do (a request) and post (an assertion to a shared data structure).

Naturally, the shortcoming of such *ad hoc* approaches is that it makes it non-trivial, if not impossible, for interoperation to occur between agent applications built by different developers. The case for having an ACL standard like KQML therefore seems compelling.

3. Languages for constructing agent applications

There are many languages currently available to prototype agent-based applications. However, these do not warrant them being referred to as agent languages, as many do. Wooldridge and Jennings [16] write:

"... by an agent language, we mean a system that allows one to program hardware or software computer systems in terms of some of the concepts developed by agent theorists. At the very least, we expect such a language to include some structure corresponding to an agent."

This is fine, but it begs the polemical question: "What is an agent?" As there is no consensus yet (indeed, there may never be) with respect to this question, it is axiomatic that a language which some researchers may refer to as an agent language will not be referred to as such by others. What is true is that there are a whole range of languages which lend themselves, to varying degrees, to different types of agent applications and definitions. It has been argued in Nwana

[2] and Nwana and Ndumu [3] that the word ‘agent’ is really a banner word for a heterogeneous body of on-going research and development, and a typology for software agents was provided. This typology is now used to classify the languages covered in this section. The typology includes the following agent types — collaborative, interface, mobile, information and reactive. More detailed information is available in Nwana [2] or Nwana and Ndumu [3].

3.1 Some languages that facilitate the building of agent applications

Table 2, which provides a list of some available languages, is by no means exhaustive; moreover, a significant number of these languages are research demonstrators only.

Table 2 Some languages for developing different agent-based applications.

Agent type(s)	Language Class	Example(s)	Major reference(s)
Collaborative agents	Actor Languages	Actors	Agha [17]
	Agent-oriented programming languages	Agent-0 Placa	Shoham [18] Thomas [19]
Interface agents	Scripting lanaguages	TCL/TK	Oustershout [20]
Information agents		Safe-TCL, Safe-Tk	http://www.smli.com/research/tct/
Mobile agents		Java	http://java.sun.com/
		Telescript	http://www.genmagic.com/
		Active Web tools	
		Python	http://www.python.org
		Obliq	http://www.research.digital.com.SRC/Obliq/
		April	McCabe [21] McCabe and Clark [22]
	Scheme-48	http://photo.net/~jar/s48.html	
Reactive agents	Reactive language	RTA/ABLE	Wavish and Graham [23]

It should also be noted that Table 2 does not show many languages as suitable for developing collaborative (multi-) agent systems. This is because, in general, collaborative agents are much more complex than interface, information

or mobile agents. They tend to require not only languages for their development, but architectures or platforms, e.g. DVMT [24] or ARCHON [25]. Alternatively, they are constructed ‘from scratch’ using some third-generation language like C++, Smalltalk, or Prolog. The languages from Table 2 are not discussed in any detail here.

3.2 Traditional languages used to construct agent applications

It is perfectly possible to implement, say, an Ada compiler in machine code. However, machine code would not generally be used for such a task, because there are better tools for the job — high-level languages like Pascal or C. In just the same way, it is possible to implement agent-based systems in languages like Pascal, C, Lisp, or Prolog. As a rule, however, one would not choose to do so because such languages are not particularly well-suited to the job. A poor choice of language will necessitate the re-implementation of what other languages offer for free. For example, it is probably not very wise to develop mobile agent applications in standard C, which allows the user to write and manipulate memory with impunity. Many people would agree that mobile agents could be malicious, and allowing arbitrary, erred access to memory is a sure way to allow trouble. Hence, writing such an application in C would require the implementation of a ‘safe’ layer on top of C, which disallows direct memory access; but doing this will involve replicating the functionality that you get with the scripting language Safe-TCL. In summary, it makes sense to use the ‘right’ (i.e. minimum cost, whatever the definition of ‘cost’ is) tool for the job. Table 2 provides a ‘first-cut’ set of several heuristics for making such choices.

Typically, object-oriented languages such as Smalltalk, Java, or C++ lend themselves more easily to the construction of agent systems. This is because the concept of an ‘agent’ is not too distant from that of an ‘object’ — agents share some properties with objects such as encapsulation, and, frequently, inheritance and message passing. However, agents differ distinctly from objects *vis-à-vis* polymorphism. Objects respond to messages by invoking certain functions within them — polymorphism ensures that they do not necessarily ‘understand’ the same messages in the same way. In contrast, agents must have a common ACL that they all understand.

4. Ontologies for agent applications

Any specific agent application is grounded in some domain ontology or ontologies. Ontology, here, means the:

“...physical study of what exists. In the AI context, ontology is concerned with which categories we can usefully quantify over and how those categories relate to each other.” [26]

In other words, it is the foundational knowledge that agents need to share enough to communicate meaningfully. Consider the scenario, borrowed from Guha and Lenat [27], of a teacher, walking into a physics class, presuming that at least the following are shared with the students:

- most of the important vocabulary to be used (e.g. time, space, causality, friction),
- most of the knowledge involving these terms in the vocabulary.

These constitute the required shared ontology between the teacher and the students for any meaningful instruction to proceed. In other specific contexts, we (as humans) also bring different ontologies to bear, e.g. when we go into restaurants; in this case, the menu lists a number of important additions to the required ontology.

Similarly, for computational agents, the common ontology contains the terms that will be used in communication between agents, and the knowledge relating to these terms. This knowledge includes their definitions, their attributes, and relationships between terms and constraints. Such a shared ontology is a *sine qua non* for any useful agent-to-agent communication using an ACL like KQML, because without it, the ACL is just syntax — much of the semantics derives from the domain ontology.

Work on ontologies in the context of agents is classified here in three different categories — *ad hoc*, ‘standard’, and global ontologies.

4.1 *Ad hoc ontologies*

As with *ad hoc* ACLs, most current agent-based prototypes have some implicitly defined ontology. It is implicit in the sense that it is imposed by the designer ‘from the outside’ — i.e. an exo-strategy is employed [27]. Therefore, an agent application in the domain of business process management, e.g. BT’s ADEPT demonstrator [28], builds the vocabulary of the business process domain into the task structures of the individual agents from the outside. Furthermore, the knowledge involving these terms is usually linked, implicitly, with the purpose (i.e. the task structures) and the co-ordination mechanisms of the agents in the demonstrator.

Naturally, there are problems with such an approach.

- Every demonstrator or application defines their own limited ontologies for their limited applications. There is little or no possibility for agent-based demonstrators based on the same domain, say business process management, to interoperate — even if they used a

standard ACL. This is because the ontologies of the two different systems would almost certainly be different.

- The approach is not scalable. For example, if the prototype is required to be extended beyond the business process management domain, say to include the domain of network management, it would require a total reconstruction of the original demonstrator in order to accommodate the two sets of ontologies.

4.2 ‘Standard’ ontologies

To counter the sort of limitations mentioned in the preceding section, there are currently efforts to develop ontologies that can be shared across disparate software developers. Perhaps the best known, most advanced work in this area is the ARPA knowledge-sharing effort (KSE), which includes the ontology-sharing project — this section focuses on this work.

The ARPA KSE program has argued for a while that a common ontology is a requirement if two or more heterogeneous agents or knowledge-based systems are to interoperate by communicating information. An ontology is required in order to counter the lack of consistency between separate knowledge bases, in terms of their different vocabulary, underlying assumptions, and, most importantly, the problem of semantics.

KIF, an acronym for Knowledge Interchange Format [29], is an ongoing effort to address such issues as semantics. Essentially, it provides an *interlingua* for knowledge bases to interoperate. In other words, for two agents with different legacy knowledge bases to interoperate, both knowledge bases could be translated into KIF, which will be the shared representation language. Given n knowledge base formats, the use of an *interlingua* such as KIF necessitates at most $2n$ format converters, as opposed to the $n(n-1)$ converters that would otherwise be required.

KIF is essentially the first-order predicate calculus, recast into a Lisp-like notation. The following example depicts the content slot of an ‘untell’ KQML performative expressed in KIF; this example is borrowed from Finin et al [10]:

```
(untell
  : language KIF
  : ontology motors
  : in-reply-to S1
  : content (= (val (torque motor1) (sim-time 5))
              (scalar 12kgf)))
```

The content slot of this message says that the torque of object motor1 at simulation time (sim-time) 5 is 12 kgf. It is

assumed that the ontology motors (mentioned in the ontology slot of the message) defines the terms torque, sim-time, kgf and so on. Note that KIF is not an implemented knowledge representation language. Rather, it is an implementation-independent *interlingua* that allows for precise and unambiguous knowledge representation. For more details, see Genesereth and Fikes [29].

Ontolingua is another ARPA-sponsored effort towards reusable ontologies. Figure 3 depicts its mode of operation.

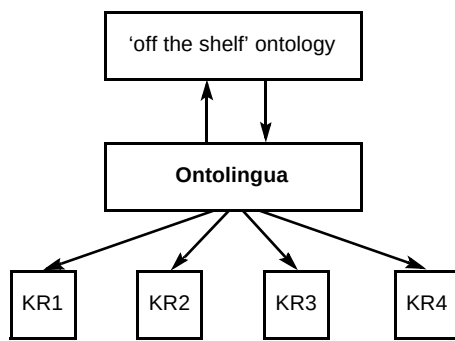


Fig 3 Ontolingua's mode of usage (abstracted from Gruber [30]).

Thus, Ontolingua is a domain-independent translation tool, which has nothing to do with the intellectual task of defining ontologies. Like KIF, it is also not an implemented language; rather, it allows for the 'explicit specification of a conceptualisation', which is how Gruber defines an ontology [30].

Ontolingua is a declarative and formal language that can be used to capture and represent ontologies at the knowledge level [31], but which acts as a mediating representation for translating from 'off-the-shelf' ontologies into several bespoke symbol-level knowledge-representation (KR) languages, e.g. Loom [32].

The knowledge sharing arises because the same Ontolingua specification, e.g. some generic knowledge-based planning program, can be translated into different symbol-level languages. The ontology of such a planner would include descriptions such as objects, events, resources, constraints, etc, and the planner assigns resources and times to objects and events [30]. Reuse occurs when the same ontology is used in different applications or by different developers.

Ontolingua statements and axioms are written in an extended KIF notation and natural language sentences, though the latter are not parsed. The Ontolingua system includes a KIF parser and syntax checker, a consistency checker, a hypertext editor and several translators from KIF to other bespoke languages. For more details, consult Gruber [30] and Ontolingua's Web page [33].

4.3 Global ontologies

There are also several efforts aimed at defining and generating more global ontologies, i.e. rather than being directed at specific domains, global ontologies are intended to be in some sense general. Such ontologies may provide invaluable substrates for agent-based applications. Two notable ones are the WordNet and the CYC projects. These are discussed briefly below.

WordNet [34] is an on-line lexical database with more than 166 000 word form and sense pairs. For example, the word form 'back' is interpreted as a noun in some linguistic context, as a verb in another, and even as an adjective or adverb in yet others. Each of these contexts are entered separately into WordNet. WordNet also includes many semantic relationships between words and word senses, and it also contains several semantic relations including synonymy, antonymy, hyponymy, meronymy, troponymy and entailment. It must be emphasised that WordNet is not really designed for agent applications; clearly, it is of much relevance to natural language understanding applications. However, it is possible to use it to complement further the ontology defined for some agent applications.

CYC [27, 35] is a project that began in 1984 with the ambitious goal of capturing and representing commonsense knowledge. The original motivation for CYC was to address the brittleness problem of expert systems. In brief, the point is that while a medical expert system might in some sense be an expert in, say, blood diseases, it will typically be unable to address any questions outside this narrow domain. CYC's inventors believe that the way to solve this problem is to build a system that has broad (though not necessarily deep) knowledge — the kind of knowledge that would be required by anyone attempting to understand an encyclopaedia article. Examples of such knowledge are:

- you have to be awake to eat,
- you can usually see people's noses but not their hearts,
- you cannot remember events that have not yet happened,
- if you cut a lump of peanut butter in half, each half is also a lump of peanut butter — but if you cut a table in half, neither half is a table.

CYC currently contains 10^5 concepts, and 10^6 commonsense axioms that have been hand-crafted into it; millions more have been inferred and cached by CYC [35]. Today, CYC's inventors believe that the first real applications of CYC will be into mainstream computing, particularly in the domain of information management [27]. Lenat [35] posits that CYC could form the standard ontology underlying the World Wide Web (WWW) and

electronic commerce. In such contexts, CYC-enabled applications which may (or may not) be agent-based, could facilitate interoperability. If the CYC dream comes true, it will obviate the need for most knowledge representation languages and current efforts including KIF and Ontolingua. For example, the content of a KQML expression could be specified in CYCL, the language in which CYC's concepts and axioms are encoded. It is unlikely that such a global ontology as CYC will be needed or available (it is currently too costly) for relatively small-scale agent demonstrators and applications in the short term. However, if CYC even approaches the functionality predicted by its inventors, it will revolutionise not just agent applications, but much of computing beyond.

5. Support computing technologies

In tandem with some developments in languages, ontologies and protocols, there are also several developments in the supporting computing technologies that facilitate the design and implementation of agent applications. Some of these are briefly highlighted below. However, to get a better appreciation of this section, perhaps a brief historical diversion is required.

Up until the mid-1980s, computing was dominated by big, monolithic mainframe applications. Such applications still exist in many large organisations. The costs that have been invested in such systems over the years, and their criticality to the smooth, everyday functioning of the organisation, ensure that their replacement — no matter how persuasive the arguments for such moves are — is anathema to the senior management of these organisations. This is the legacy software problem and, literally, it gets worse by the day.

The idea of client/server computing was proposed as an alternative to the centralised computing model more than two decades ago. Since then, client/server computing has prompted a Kuhnian paradigm shift in the computing industry. However, the idea only began to really change computing in the 1980s, with the development of better hardware, and the demand for applications that matched this hardware. The new hardware included personal computers (PCs) and local area networks (LANs) such as Ethernet. Client/server computing changed computing by replacing monolithic mainframe applications, which were typically accessed via green-screen terminals attached directly to these mainframes.

Today, clients, which are typically PCs with sophisticated graphical user interfaces (GUIs), interact with server programs that manage shared resources; server applications typically manage databases.

5.1 Agents and client/server computing

The key point here is that the client/server model naturally lends itself to the implementation of software agent systems. To see why, first note that there are several motivations for having multiple agent systems. They include [2]:

- to solve problems that are too large for a centralised single agent to do due to resource limitations or the sheer risk of having one centralised system,
- to allow for the interconnecting and interoperation of multiple existing legacy systems such as expert systems and decision-support systems,
- to provide solutions to inherently distributed problems such as distributed sensor networks [24] or air-traffic control,
- to enhance modularity (which reduces complexity), speed (due to parallelism), reliability (due to redundancy), flexibility (i.e. new tasks are composed more easily from the more modular organisation) and reusability at the knowledge level (hence shareability of resources).

These points all relate to the merits of decentralised computing over monolithic systems — agent-based systems are decentralised systems. The client/server model thus closely matches the decentralisation associated with agent technology. To summarise, there is a synergistic relationship between agent technology and client/server computing, and hence client/server architectures provide a natural environment within which to develop agent applications.

5.2 Agents and distributed objects

Although we are yet to see the full impact of the first client/server revolution, advances in hardware are already fuelling a second client/server revolution [36]. This second revolution is being driven by wide area networks (WANs) and object-oriented technology. The integration of these two relatively old technologies yields a powerful new approach for achieving large-scale client/server computing — distributed objects. As Orfali et al write:

“...by and large, today's client/server applications remain difficult to build, manage, and extend. Distributed objects change this. With the proper packaging and infrastructure, objects will help subdivide today's monolithic client/server applications into self-managing components that can play together and roam across networks and operating systems. Component-like objects allow us to create client/server

systems by assembling ‘live blobs of intelligence and data’ in an infinite number of lego-like arrangements. These components represent the ultimate form of client/server distribution and prepare us for the near future when millions of machines — mostly desktops — will be both clients and servers.” [36]

Distributed objects allow for the packaging of software into components with well-defined interfaces which, in turn, offers other advantages. Firstly, new information systems can be built by assembling various components, lego-like, as long as the interfaces are consistent with one another. Secondly, components or objects are portable — they can run on different operating systems such as OS/2, Macintosh or Unix. In 1989, a consortium of object vendors grouped together to form the Object Management Group (OMG).

Since then, they have been defining standards and an architecture (shown in Fig 4), which allows object components written by different vendors to interoperate across networks and operating systems. Hence, interoperability is a third advantage of distributed objects. The key to interoperability is the object bus, or Object Request Broker (ORB). The ORB allows clients (objects) to make requests and receive responses, statically or dynamically, to other local objects, which act as servers. Clients are totally unaware of the lower-level communication mechanisms involved. The specification of the architecture is referred to as the Common Object Request Broker Architecture (CORBA), and the first set of specifications, CORBA 1.1, were agreed in 1991. In December 1994, the CORBA 2.0 standard was agreed, which allows for interoperability across ORBs from different vendors.

The key to CORBA is its Interface Definition Language (IDL). The IDL is a declarative language, which allows for the definition of application program interfaces (APIs). The IDL definition language allows for the definition of the

objects, their attributes, the methods they export and the parameters of these methods. IDL has no control structures or variables, and, together with the ORB, constitutes the object bus.

Objects written in different programming languages, packaged as binaries, and running on different operating systems, can have IDL interfaces so that they can interoperate, and invoke each others’ methods. The point of this is that objects written in, say, Cobol, Lisp, or C++ linked to an ORB, can now interoperate in various client/server modes across different operating systems. Clients do not therefore need to care about where the distributed objects reside nor on what operating systems they are running.

This is a significant development, even within client/server computing. For example, it would allow work-flow objects on some server in Amsterdam to interoperate with some customer’s business process objects in Ipswich. Therefore, a fourth advantage of distributed objects is that new objects may co-exist with legacy applications, if the latter are provided with IDL interfaces. OMG’s ultimate goal is to create a collaborative client/server component-based environment.

What has all these got to do with agents? Simply that such emerging technologies and standards ensure that computing will continue to be increasingly decentralised. Such decentralised architectures, as noted earlier, provide natural substrates for the construction of an agent layer. This agent layer will provide many of the core services required to build agent applications. This is already happening — the ADEPT demonstrator [28] has essentially built such an agent layer on top of a CORBA-compliant implementation.

6. Discussion and conclusions

This closing section briefly discusses some of the issues that have emerged in this survey. As indicated at several points in this paper, standards play an important role in

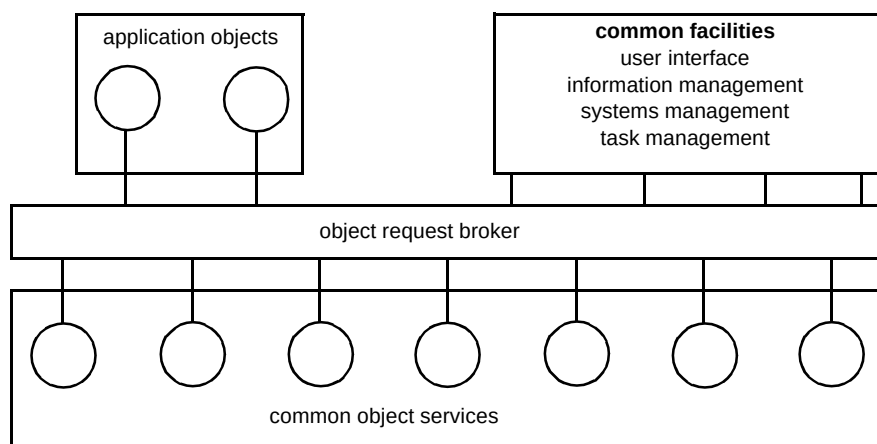


Fig 4 The OMG object management architecture (adapted from Orfali et al [36])

agent-based systems, for exactly the same reason that they are important in distributed systems and telecommunications generally — agents need to be able to talk to, and understand one another. Yet while the importance of standards seems clear, there are also a number of dangers associated with standardisation. It is worth pausing to consider what these dangers are, and how they might adversely affect the development of agent systems.

Firstly, there is the danger of standardising too early, or, equivalently, of picking the wrong standard. Arguably, standards are most successful in stable domains, or domains where the future development of the technology is easy and safe to predict. This is not the case with the agent field. To see why, consider the World Wide Web. This enormously popular service did not exist before 1993; nor could its enormous success have been predicted. If anyone attempted to standardise information services for WANs in 1992, they were almost certainly wasting their time. Unfortunately, the agent field is developing at a similarly rapid pace. This makes standardisation a particularly difficult problem for agent researchers. It could be argued that KQML (the agent communication language discussed in detail in section 2.1) is an attempt to standardise too early, as the requirements for ACLs are not sufficiently understood — there are too few real multi-agent systems in existence to know what features an ACL must support.

Just as it is possible to standardise too early, so it is possible to standardise too late. A good example here is the X.400 e-mail standard, which was widely expected to take over from the Internet's Simple Mail Transfer Protocol (SMTP). This did not happen, despite the technical superiority of X.400 over the cruder (and comparatively ancient) SMTP standard. One reason for this failure was simply that by the time X.400 had passed through the time-consuming international standardisation process, and working X.400 systems became available, SMTP was already established as the *de facto* standard. So, although X.400 was technically right (in that it provided for many sophisticated and powerful services) it arrived too late to have the impact it perhaps deserved. A similar situation within the agent field may be occurring with Telescript [37] a programming language and development environment for mobile agent applications. Telescript was eagerly awaited within the Internet agent community, but by the time it was made available to the public, the Java programming language had already been released, and was supported within the popular Netscape World Wide Web browser. As a result, Java had a very large end-user base, making it much more attractive to software developers than Telescript. Java does not, however, provide facilities for building mobile agent applications in the way that Telescript does. In this sense, the two languages are not really competing — but Java appears to be winning the race anyway.

Standards will be essential if agent technology is to realise its potential; but, as it is hoped has been made clear, standardisation is as much about timing and judgement as it is about technical issues.

In summary, no firm predictions can be made about exactly what form future agent-based systems will take, but it is certain that agents have much to offer. It is hoped that this paper has illustrated some of the technologies that are likely to underpin this promising new software industry.

Acknowledgements

Discussions with Divine Ndumu and Mark Wiegand, that have shaped some of the ideas propounded in this paper, are gratefully acknowledged.

References

- 1 Mayfield M, Labrou Y and Finin T: 'Evaluation of KQML as an Agent Communication Language', in Wooldridge M, Mueller J P and Tambe M (Eds): 'Intelligent Agents II', Lecture Notes in Artificial Intelligence 1037, Heidelberg, Springer Verlag (1996).
- 2 Nwana H S: 'Software Agents: An Overview', The Knowledge Engineering Review, 11, No 3 (1996).
- 3 Nwana H S and Ndumu D T: 'An introduction to agent technology', BT Technol J, 14, No 4, pp 55—67 (October 1996).
- 4 Nwana H S, Lee L, and Jennings N R: 'Co-ordination in software agent systems', BT Technol J, 14, No 4, pp 79—88 (October 1996).
- 5 Genesereth M R and Ketchpel S P: 'Software Agents', Communications of the ACM, 37, No 7, pp 48—53 (1994).
- 6 Cohen P R and Levesque H J: 'Communicative Actions for Artificial Agents', in Proceedings of the First International Conference on Multi-Agent Systems, San Francisco, Cambridge, AAAI Press, pp 65—72 (1995).
- 7 Austin J L: 'How to Do Things with Words', Cambridge, MA, Harvard University Press (1962).
- 8 Searle J R: 'Speech Acts', Cambridge, MA, Cambridge University Press (1969).
- 9 Neches R: 'Overview of the DARPA Knowledge Sharing Effort', on <http://www-ksl.stanford.edu/knowledge-sharing/papers/kse-overview.html> (1994).
- 10 Finin T, Fritzson R, McKay D and McEntire R: 'KQML as an agent communication language', in Proceedings of the 3rd International Conference on Information and Knowledge Management (CIKM), New York, ACM Press, also available from <http://www.cs.umbc.edu/kqml/papers/> (November 1994).
- 11 Labrou Y and Finin T: 'A Semantics Approach for KQML — a general purpose communication language for software agents', in Proceedings of the 3rd International Conference on Information and Knowledge Management (CIKM), New York, ACM Press, also available from <http://www.cs.umbc.edu/kqml/papers/> (November 1994).
- 12 Finin T: 'Agent Communication Languages: KQML and the Knowledge Sharing Effort', abstract, <http://umbc.edu/~finin> (1995).
- 13 Finin T, Weber J, Wiederhold G, Genesereth M, Fritzson R, McKay D, McGuire J, Pelavin R, Shapiro S and Beck C: 'Specification of the KQML Agent-Communication Language', Draft (February 1994).

- 14 Huhns M M: 'KQML', Tutorial Notes, Tutorial F, Multi-Agent Systems Tools & Research Methods, International Conference on Multi-Agent Systems, San Francisco (1995).
- 15 Cohen P R, Cheyer A, Wang M and Baeg S C: 'An Open Agent Architecture', in Proceedings of AAAI Spring Symposium, California, AAAI Press (March 1994).
- 16 Wooldridge M and Jennings N: 'Intelligent Agents: Theory and Practice', The Knowledge Engineering Review, 10, No 2, pp 115—152 (1995).
- 17 Agha G: (1986), Actors: 'A Model of Concurrent Computation in Distributed Systems', London, MIT Press (1986).
- 18 Shoham Y: 'Agent-Oriented Programming', Artificial Intelligence, 60, No 1, pp 51—92 (1993).
- 19 Thomas S R: 'The PLACA Agent Programming Language', in Wooldridge M and Jennings N R (Eds): 'Intelligent Agents — Theories, Architectures, and Languages', Lecture Notes in Artificial Intelligence 890, Heidelberg, Springer Verlag (1995).
- 20 Oustershout J K: 'Tcl and the Tk Toolkit', New York, Addison Wesley (1994).
- 21 McCabe F G: 'APRIL Reference Manual', Version 2.1, Department of Computer Science, Imperial College, London (1995).
- 22 McCabe F G and Clark K L: 'APRIL — Agent PProcess Interaction Language', in Wooldridge M and Jennings N R (Eds): 'Intelligent Agents — Theories, Architectures, and Languages', Lecture Notes in Artificial Intelligence 890, Heidelberg, Springer Verlag (1995).
- 23 Wavish P and Graham M: 'A Situated Action Approach to Implementing Characters in Computer Games', Applied AI Journal, 10, No 1, pp 53—74 (1996).
- 24 Durfee E H, Lesser V R and Corkill D: 'Coherent Cooperation among Communicating Problem Solvers', IEEE Transactions on Computers, C-36 No 11, pp 1275—1291 (1987).
- 25 Jennings N, Corera J M, Laresgoiti L, Mamdani E, Perriollat F, Skarek P and Varga L: 'Using ARCHON to Develop Real-World DAI Applications for Electricity Transportation and Particle Accelerator Control', IEEE Expert, Special Issue on Real-World Applications of DAI systems (1995).
- 26 Rich, E and Knight, K. 'Artificial Intelligence (Second Edition)', New York, McGraw Hill (1991).
- 27 Guha R V and Lenat D B: 'Enabling Agents to Work Together', Communications of the ACM, 37, No 7, pp 127—142 (1994).
- 28 O'Brien P D and Wiegand M E: 'Agents of Change in Business Process Management', BT Technol J, 14, No 4, pp 133—140 (1996).
- 29 Genesereth M R and Fikes R E (Eds): 'Knowledge Interchange Format', Version 3.0 Reference Manual, Computer Science Department, Stanford University, Technical Report Logic-92-1, also available from <http://www-ksl.stanford.edu/knowledge-sharing/kif/> (March 1992).
- 30 Gruber T R: 'A Translation Approach to Portable Ontology Specifications', Knowledge Acquisition, 5, No 2, pp 199—220 (1993).
- 31 Newell A: 'The Knowledge Level', Artificial Intelligence, 18, pp 87—127 (1982).
- 32 MacGregor R: 'The Evolving Technology of Classification-Based Knowledge Representation Systems', in Sowa J (Ed): 'Principles of Semantic Networks: Explorations in the Representation of Knowledge', San Mateo, CA, Morgan Kaufmann (1991).
- 33 Ontolingua: <http://www-ksl.stanford.edu/knowledge-sharing/ontolingua/>
- 34 Miller G A: 'WordNet: A Lexical Database for English', Communications of the ACM, 38 No 11, pp 39—41 (1995).
- 35 Lenat D B: 'CYC: A Large-Scale Investment in Knowledge Infrastructure', Communications of the ACM, 38, No 11, pp 33—38 (1995).
- 36 Orfali R, Harkey D and Edwards J: 'The Essential Distributed Object Survival Guide', New York, John Wiley & Sons (1996).
- 37 White J E: 'Telescript Technology: The Foundation for the Electronic Market-place', White Paper, General Magic Inc, 2465 Latham Street, Mountain View, CA 94040 (1994).



Hyacinth S Nwana holds a BSc in Computer Science and Electronic Engineering (1985), an MSc. in Computer Science (1986) and a PhD in the area of Artificial Intelligence (1989). Between 1989 and 1995, he worked in various roles (lecturer, researcher, contract and visiting scientist) for the Universities of Liverpool, Keele, Calgary (sponsored by the Royal Society), Shell Research Laboratories and BT Laboratories. He joined BT in October 1995 and leads part of BT's software agents research programme, as well as a team researching into the engineering of agent-based systems and is contactable on e-mail at: hyacinth@info.bt.co.uk. He has a broad range of AI-related interests and has written on software agents, AI and education, distributed AI, machine learning, neural networks, knowledge acquisition, computer science education and cognitive science. In 1991, he principally won the Digital Equipment Corporation (DEC) European Artificial Intelligence prize. He is currently pursuing a part-time MEd degree at Cambridge. He is a member of the British Computer Society and a Chartered Engineer.



Michael Wooldridge gained a first-class honours degree in computer science in 1989, and went on to study for a PhD, which he obtained from the University of Manchester (UMIST) in 1992. His doctoral thesis was in the theory of multi-agent systems. Between September 1992 and September 1996, he was employed as a lecturer in the Department of Computing at Manchester Metropolitan University, where he continued his research into multi-agent systems, focusing in particular on the relationship between formalisms used to reason about agents and concrete realisations of such agents. Since October 1996, he has been employed as a member of the agents team at Mitsubishi Electric Digital Libraries group in London, carrying out research and development of real-world agent-based systems (contactable on e-mail: mjw@dlib.com). He is the editor of two successful books ('Intelligent Agents', Springer-Verlag, 1995, and 'Intelligent Agents II', Springer-Verlag, 1996), and has published over thirty refereed articles in the theory and practice of agent-based systems.

